

Universidad Tecnológica Nacional.
Tecnicatura Universitaria en Programación. 1Pro1.
21 de junio del 2026.
Programación II.
Docente Naveda.
Alumnos Gabriel D'anna, Gonzalo Sánchez,
Danilo Serrano y Jesús Ramírez.

Trabajo práctico integrador: sistema de gestión de pedidos de comida.

Índice.

Índice.....	2
Marco teórico.....	3
Decisiones técnicas y arquitectura.....	5
Dificultades y soluciones.....	6
Capturas de ejecución.....	7
Análisis del problema.....	8

Marco teórico.

Primero que nada pasamos a las tecnologías que hicieron posible este proyecto.

- **Java:** es el lenguaje de programación utilizado para la aplicación. Es reconocido por su estabilidad y por poder ejecutarse en prácticamente cualquier computadora. Se ciñe a la programación orientada a objetos.
- **GitHub:** es una plataforma en la nube que funciona como historial detallado del proyecto. Permite guardar el código y sus distintas versiones de forma segura. Facilita el trabajo en equipo, evitando que los cambios de una persona sobrescriban los de otra.
- **JDK 21:** es el kit de desarrollo de Java (versión 21) necesario para escribir, traducir y hacer funcionar las aplicaciones creadas en Java.
- **Clase:** es una estructura o plantilla fundamental en la Programación Orientada a Objetos que actúa como un plano a nivel de código fuente que define la estructura de datos (estado) y las operaciones (comportamiento) que tendrán los objetos instanciados a partir de ella.
- **Objeto:** es una instancia concreta de una clase en tiempo de ejecución.
- **Atributo:** es una variable declarada dentro de una clase que define su estado interno.
- **Método:** es una subrutina o función definida dentro de una clase que establece el comportamiento ejecutable del objeto.
- **Colecciones:** son arquitectura de interfaces (List, Set, Map) y sus implementaciones para la manipulación de estructuras de datos en memoria.
- **Excepciones:** son mecanismos de control de flujo que previenen la interrupción súbita del código cuando ocurre un error inesperado. Permiten que el sistema controle los distintos problemas que podrían aparecer al momento de la ejecución.
- **Enums:** son tipos de datos que representan un conjunto fijo de constantes, aportando seguridad de tipos en tiempo de compilación.

- **Clases abstractas:** son tipos base no instanciables dentro de una jerarquía de herencia que permiten la definición de métodos concretos y abstractos.
- **Interfaces:** son plantillas que permiten definir métodos predefinidos para que otras clases afines puedan implementar, permitiendo la herencia múltiple de tipos y fomentando el bajo acoplamiento.
- **Bases de datos relacionales:** son sistemas de gestión que almacenan datos en tablas normalizadas, garantizando integridad referencial y propiedades ACID.
- **SQLite:** es un sistema de gestión de bases de datos relacionales embebido, autocontenido y sin servidor, que almacena la base de datos en un único archivo de disco.
- **JDBC:** es una API para Java que proporciona interfaces para la conexión y ejecución de sentencias SQL en bases de datos relacionales.

Ahora tenemos que explicar algunos conceptos claves de la programación:

- **Encapsulamiento:** refiere el ocultamiento del estado interno de un objeto mediante modificadores de acceso, restringiendo la interacción externa exclusivamente a través de miembros públicos controlados.
- **Abstracción:** es el proceso de ocultar los detalles de implementación y mostrar en su lugar únicamente los detalles esenciales de un objeto.
- **Herencia:** es un mecanismo estructural mediante el cual una subclase hereda atributos y métodos de una superclase, reutilizando código bajo una relación de tipo *es un*.
- **Polimorfismo:** refiere la capacidad de un objeto o método para comportarse de forma distinta dependiendo del contexto.
- **Cohesión:** es la medida del grado en que los elementos dentro de un único módulo o clase están funcionalmente relacionados y enfocados en un propósito único. Es, idealmente, alto.
- **Acoplamiento:** es la medida del grado de interdependencia o nivel de conocimiento directo que tiene un módulo, clase o componente sobre otros. Es, idealmente, bajo.
- **Relaciones:** Vinculaciones estructurales y semánticas entre clases que determinan la interacción y dependencia de los objetos dentro del modelo de dominio.

Arquitectura del proyecto.

El proyecto es un sistema de gestión de pedidos de comida que opera con base de datos mediante la API Java Database Connectivity (JDBC) bajo el patrón de diseño Data Access Object (DAO). El JDBC consigue crear y gestionar una base de datos local gracias al empleo de SQLite. A continuación se desglosa su composición, dividida en distintos paquetes.

config.

Contiene únicamente la clase `DatabaseConfig`, donde se hallan los métodos que hacen posibles la creación y conexión con una base de datos.

Para conseguir conectarse a una base de datos (o crear una si no se la tiene ya), se creó el método `conectar()`, que a su vez emplea el método `getConnection(String url)` de la clase `DriverManager` proveída por la API de SQL. Se le pasa un `String` y el retorno del método se le adjudica al objeto `con`, que es el retorno de `conectar()`. Éste es un método empleado 23 veces, puesto que cada vez que se tiene que comunicar de alguna forma con la base de datos, se usará.

En la misma clase `DatabaseConfig` se encuentran también métodos para la creación (si no se las dispone ya) de las tablas SQL para usuarios, productos, pedidos, categorías y detalles SQL, en los cuales cabe destacar el uso de objetos `Statement`, que serán utilizados para ejecutar código SQL estático y devolver sus resultados.

dao.

Este paquete contiene las clases que nos servirán de DAOs: `CategoriaDao`, `PedidoDao`, `ProductoDao`, `UsuarioDao`; es decir, contiene las clases que mediarán entre código y base de datos. Cada vez que se necesita añadir, leer, actualizar o *eliminar* (los objetos no se borran, sólo se registran como eliminados) datos a una tabla, se emplea la clase `dao` pertinente.

Éstas clases usan métodos estáticos para ahorrarnos instancias reiteradas e innecesarias. Todos poseen (reemplazando los puntos suspensivos por el género que les corresponde) métodos `insertar...()`, `obtenerTodos()`, `editar...()` y `eliminar...()` que nos permiten realizar las operaciones CRUD cruciales tanto para el proyecto como el sistema.

entities.

Presenta Base, un objeto abstracto; Calculable, una interfaz; y las clases Categoria, DetallePedido, Pedido, Producto y Usuario, que heredan de Base y, en el caso de detallePedido, implementan Calculable.

Base contiene características compartidas por todas las clases que se quieren implementar como objetos que serán añadidos a las tablas de la base de datos: id, eliminado y createdAt; a su vez, contiene getters y un método delete() que se encarga de marcar el objeto como eliminado mediante el cambio del valor de su atributo eliminado (nuevamente, no se elimina nada). A parte de esos atributos, las clases que heredan de Base también poseen sus propios atributos y métodos convenientes.

enums.

Contiene Estado, FormatoPago y Rol, enums conveniente para el establecimiento de datos específicos dentro de otras partes del código.

exceptions.

Contiene dos clases de excepciones, CantidadInvalidaException y ListaDetallePedidoException, cuyas funciones son autoexplicativas; a su vez, contiene la clase ValidacionesDeEntrada, que contiene métodos estáticos que permiten inputs controlados y prevenidos contra errores, evitando así la reiteración innecesaria de código.

menus.

Contiene MenuCategoria, MenuPedido, MenuProducto y MenuUsuario, clases a las que será enrutado el usuario desde algún menú de la clase Main (situada en la carpeta java, fuera de los paquetes). Éstas se encargan de proveer un ambiente visual por consola al usuario, y de hacer consultas a los Daos para así conseguir comunicar con la base de datos al comando del usuario. Siguen las operaciones CRUD (create, read, update and delete).

Dificultades y soluciones.

Antes de dar los primeros pasos en el desarrollo, nos encontrábamos en un estado de notable incertidumbre, bastante perdidos respecto a cómo abordar la persistencia de los datos de manera eficiente sin sobrecargar la aplicación de consola. Esta encrucijada inicial se resolvió al decantarnos por el empleo de SQLite a través de la API JDBC, lo que nos proporcionó un entorno de base de datos local muy accesible y directo para empezar a construir. Lograr una primera conexión con la base de datos nos dio ánimo e hizo parecer posible el proyecto.

No obstante, a medida que el sistema comenzó a expandirse, la escala real del proyecto resultó ser un desafío conceptual importante, principalmente por la dificultad de conciliar en la mente la interconexión de tantas partes necesarias para llevarlo a cabo. Resultaba complejo visualizar con claridad matemática cómo debían coexistir y dialogar las entidades del negocio con las clases mediadoras de la base de datos y con los menús interactivos que se le presentan al usuario final.

En este escenario de crecimiento, otra de las grandes dificultades técnicas fue tener presente la enorme cantidad de métodos requeridos y el control estricto sobre las funciones específicas de cada uno, corriendo constantemente el riesgo de duplicar lógica o confundir las responsabilidades entre capas.

Logramos subsanar estos inconvenientes mediante una aplicación rigurosa de los pilares de la programación orientada a objetos y una estructuración limpia en paquetes independientes. El diseño estratégico de la clase abstracta Base para centralizar atributos comunes, sumado a la implementación de métodos estáticos tanto en los DAOs como en las validaciones de entrada, funcionó como un excelente anclaje organizativo. Esto no solo nos ahorró instancias reiteradas e innecesarias a lo largo del código, sino que también estandarizó los comportamientos del sistema, permitiéndonos mantener el control absoluto del flujo y la lógica sin perder el rumbo dentro de la arquitectura del proyecto.

Capturas de ejecución.

```
- conexion exitosa -
[DATABASE]: Tabla Usuarios creada.
- conexion exitosa -
[DATABASE]: Tabla Productos creada.
- conexion exitosa -
[DATABASE]: Tabla Pedidos creada.
- conexion exitosa -
[DATABASE]: Tabla Categorías creada.
- conexion exitosa -
[DATABASE]: Tabla Detalles creada.
=== SISTEMA DE PEDIDOS (FOOD STORE) ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Ingrese una opcion: .
```

```
Seleccione una opcion: 1
- Lista de categorías -
1. [ID: a78a55b8-6adf-4933-a01d-c1cb08b604af] Fiambres - Queso, jamón y esas cosas
-----
-- GESTION DE CATEGORIAS --
1. Listar categorías
2. Crear categoría
3. Editar categoría
4. Eliminar categoría
0. Salir
Seleccione una opcion: 4
- Eliminar categoría existente -
1. Fiambres
Seleccione una categoría: 1
¿Confirma la eliminación de 'Fiambres'? (S/N): s
- conexion exitosa -
[BD]: Categoría eliminada.
```

```
Seleccione una opcion: 2
- conexion exitosa -
- Creacion nueva categoría -
Nombre de la categoría: Fiambre
Descripcion de la categoría: Comida curada y conservada al frío para su consumo posterior
- conexion exitosa -
[BD]: Subida de categoría Fiambre exitosa!
Categoría creada con ID: a78a55b8-6adf-4933-a01d-c1cb08b604af
- conexion exitosa -
-- GESTION DE CATEGORIAS --
1. Listar categorías
2. Crear categoría
3. Editar categoría
4. Eliminar categoría
0. Salir
Seleccione una opcion: 1
- Lista de categorías -
1. [ID: a78a55b8-6adf-4933-a01d-c1cb08b604af] Fiambre - Comida curada y conservada al frío para su consumo posterior
```

```
Seleccione una opcion: 3
1. Fiambre
Seleccione una categoría: 1
- Edicion categoría existente -
Nombre de la categoría: Fiambres
Descripcion de la categoría: Queso, jamón y esas cosas
- conexion exitosa -
[BD]: Categoría con ID a78a55b8-6adf-4933-a01d-c1cb08b604af actualizada con éxito.
Categoría actualizada correctamente.
- conexion exitosa -
-- GESTION DE CATEGORIAS --
1. Listar categorías
2. Crear categoría
3. Editar categoría
4. Eliminar categoría
0. Salir
Seleccione una opcion: 1
- Lista de categorías -
1. [ID: a78a55b8-6adf-4933-a01d-c1cb08b604af] Fiambres - Queso, jamón y esas cosas
```

Bibliografía.

1. [About the Java Technology](#)
2. [About GitHub and Git - GitHub Docs](#)
3. [Java SE Technologies - Database](#)
4. [JDK 21 Readme](#)
5. [Class \(Java Platform SE 8\)](#)
6. <https://dev.java/learn/oop/>
7. <https://www.geeksforgeeks.org/java/java-collection-tutorial/>
8. <https://docs.oracle.com/javase/tutorial/essential/exceptions/definition.html>
9. <https://dev.java/learn/classes-objects/enums/>
10. <https://docs.oracle.com/javase/tutorial/java/IandI/abstract.html>
11. <https://docs.oracle.com/javase/tutorial/java/concepts/interface.html>
12. <https://www.ibm.com/mx-es/think/topics/relational-databases>
13. <https://sqlite.org/about.html>
14. <https://docs.oracle.com/javase/8/docs/technotes/guides/jdbc/>
15. <https://docs.oracle.com/javase/tutorial/jdbc/basics/index.html>
16. <https://prgrmmng.com/coupling-and-cohesion-java>

Link del repositorio de GitHub.

[Gonza-12-Sanchez/FoodStore-SistemaDeGestionDePedidosDeComida-Grupo9.](#)